

RankEX — User Stories

Estrate dal codice sorgente · Versione maggio 2026 · 95 US totali
Formato espanso: ogni US include storia, contesto e file di riferimento.

Autenticazione

US-001 — Login

Come utente non autenticato **voglio** inserire email e password **per** accedere alla piattaforma in base al mio ruolo.

Il form autentica tramite Firebase Auth. Al successo il sistema carica il profilo da `/users/{uid}`, legge il ruolo e reindirizza alla view corretta: `trainer/org_admin/staff_readonly` → `TrainerShell`, `client` → `ClientView`, `super_admin` → `AdminShell`. Se il flag `mustChangePassword` è `true`, il client viene bloccato su `ChangePasswordScreen` prima di raggiungere la dashboard. Gli errori Firebase (credenziali errate, utente disabilitato) vengono tradotti in messaggi leggibili in italiano.

`features/auth/LoginPage.jsx`

US-002 — Reset password via email

Come utente registrato **voglio** richiedere il reset della password via email **per** recuperare l'accesso se la dimentico.

Firebase invia un link di reset all'indirizzo inserito. Dopo l'invio compare una schermata di conferma. La funzione è accessibile anche ai client che hanno perso la password temporanea, senza richiedere autenticazione attiva.

`features/auth/components/ResetForm.jsx`

US-003 — Cambio password obbligatorio al primo accesso

Come client che accede per la prima volta **voglio** essere forzato a cambiare la password temporanea **per** proteggere il mio account da subito.

Quando il trainer crea un cliente, il flag `mustChangePassword: true` viene scritto in `/users/{uid}`. Al login, `ClientView` intercetta il flag e mostra `ChangePasswordScreen` prima della dashboard. Il client deve inserire la password temporanea corrente (re-auth) e scegliere una nuova password che rispetti la policy (min 8 caratteri, almeno 1 numero, 1 maiuscola). Solo dopo il cambio il flag viene azzerato a `false` e la dashboard diventa accessibile.

`features/client/ChangePasswordScreen.jsx`

US-004 — Logout automatico per inattività

Come utente autenticato **voglio** essere disconnesso automaticamente dopo un periodo di inattività **per** garantire la sicurezza della sessione.

`useSessionTimeout(role)` ascolta eventi `mousemove`, `keypress`, `touchstart`, `scroll`. Se l'intervallo scade senza attività, esegue il logout. Le soglie variano per ruolo: `super_admin` 30 min, `org_admin` 2 ore, `trainer/staff` 8 ore, `client` 24 ore. Protegge sessioni dimenticate aperte su dispositivi condivisi o postazioni pubbliche.

`hooks/useSessionTimeout.js`

US-005 — Logout esplicito

Come utente autenticato **voglio** fare logout esplicito **per** terminare la sessione in sicurezza.

Il logout chiama `signOut()` di Firebase Auth preceduto da `auditLog(AUDIT_ACTIONS.LOGOUT)` per garantire tracciabilità. La sessione viene terminata lato client; per limitazioni strutturali del piano Spark senza backend, il refresh token Firebase mantiene una residua autonomia di circa 1 ora.

```
firebase/services/auth.js
```

Gestione Clienti (Trainer)

US-006 — Lista clienti

Come trainer **voglio** visualizzare la lista di tutti i miei clienti **per** avere una panoramica del gruppo di lavoro.

Mostra tutti i clienti dell'organizzazione in una griglia di card. Ogni card riporta nome, categoria o ruolo, rank, livello XP e badge fascia (per Soccer). Supporta ricerca testuale e ordinamento per nome, rank o data aggiunta. La lista è paginata e filtrata tramite `FiltersSidebar`.

```
features/trainer/ClientsPage.jsx
```

US-007 — Filtro clienti per categoria / ruolo / fascia

Come trainer **voglio** filtrare i clienti per categoria, ruolo o fascia **per** trovare rapidamente i clienti con un profilo specifico.

In modulo PT filtra per categoria (health / active / athlete). In Soccer Academy filtra per ruolo calcistico (portiere / difensore / centrocampista / attaccante) e per fascia d'età (Pulcini / Esordienti / Senior). Il filtro FASCIA appare solo quando nell'organizzazione sono presenti clienti appartenenti ad almeno due fasce diverse.

```
features/trainer/clients-page/FiltersSidebar.jsx
```

US-008 — Crea nuovo cliente tramite wizard

Come trainer **voglio** aggiungere un nuovo cliente tramite un wizard guidato **per** raccogliere anagrafica, profilo e credenziali in un unico flusso.

Il wizard multi-step raccoglie dati anagrafici, categoria o ruolo, tipo profilo, eventuali misure BIA iniziali e genera le credenziali Firebase Auth. Se l'organizzazione ha raggiunto il limite clienti del piano, viene mostrata una schermata di blocco al posto del wizard.

```
features/trainer/NewClientView.jsx
```

US-009 — Elimina cliente

Come trainer **voglio** eliminare un cliente con conferma esplicita **per** rimuovere profili non più attivi.

Un `ConfirmDialog` a due step (conferma generica, poi digitazione del nome) impedisce eliminazioni accidentali. L'eliminazione usa un batch Firestore: cancella il documento cliente, decrementa atomicamente `clientCount` sull'organizzazione e registra `AUDIT_ACTIONS.CLIENT_DELETED` nell'audit log.

```
hooks/useClients.js
```

Dashboard Cliente — Vista Trainer

US-010 — Scheda atleta (avatar, nome, badge, XP)

Come trainer **voglio** visualizzare la scheda atleta del cliente **per** avere un riepilogo immediato del suo stato.

Il pannello sinistro della dashboard (sticky su desktop, tab AVATAR su mobile) mostra un avatar placeholder colorato con il colore del rank, i badge di categoria/ruolo/fascia, la XPBar orizzontale fullWidth e il livello corrente. Su mobile il pannello si trova nel primo tab (AVATAR), nascosto su desktop.

```
features/client/ClientDashboard.jsx
```

US-011 — Rank e media percentile

Come trainer **voglio** vedere il rank e la media percentile del cliente **per** valutare il livello atletico complessivo.

Il rank è calcolato da `getRankFromMedia(media)` dove `media` è la media aritmetica dei percentili di tutti i test campionati. Viene visualizzato come etichetta testuale (IRON / BRONZE / SILVER / GOLD / PLATINUM / DIAMOND) con il colore associato. La BIA non influenza il rank atletico.

```
features/client/ClientDashboard.jsx
```

US-012 — Navigazione tab dashboard

Come trainer **voglio** navigare tra le tab della dashboard **per** accedere alle diverse sezioni operative.

La tab nav è sticky sotto l'header. Le tab disponibili sono: Panoramica, Test, BIA, Note, Misure, Scheda, XP Trend, Sessioni e (solo mobile) AVATAR. Il tab AVATAR è nascosto su desktop via `lg:hidden`. Lo staff_readonly vede tutte le tab in sola lettura senza controlli di modifica.

```
features/client/ClientDashboard.jsx
```

US-013 — Campionamento test

Come trainer **voglio** eseguire un campionamento e registrare i valori misurati **per** aggiornare il profilo atletico del cliente.

`CampionamentoView` presenta i test della categoria o fascia del cliente. Ogni campo accetta il valore misurato; le formule complesse (es. Y Balance composite con entrambi gli arti) vengono applicate automaticamente. Il campionamento è associato alla data odierna.

```
features/client/CampionamentoView.jsx
```

US-014 — Percentili in tempo reale durante il campionamento

Come trainer **voglio** vedere i percentili calcolati in tempo reale durante il campionamento **per** avere feedback immediato sulla prestazione.

Ad ogni modifica di un campo, `calcPercentileEx` ricalcola il percentile e aggiorna la visualizzazione live. Se l'età del cliente è fuori dalla fascia normativa disponibile, compare un banner ambra sopra il campo (`ageWarning`): il percentile viene comunque stimato dalla fascia normativa più vicina, ma il trainer è avvisato dell'incertezza.

```
features/client/CampionamentoView.jsx
```

US-015 — Salva campionamento con aggiornamento XP e rank

Come trainer **voglio** salvare il campionamento con aggiornamento XP e rank **per** mantenere lo storico delle performance nel tempo.

`saveCampionamentoUseCase` calcola l'XP assegnato in base al numero di stat percentili migliorate (tier: FIRST 50 / NONE 10 / PARTIAL 30 / MOST 60 / ALL 100). Aggiorna in un'unica write Firestore: `stats` , `media` , `rank` , `campionamenti[]` , `log[]` , `xp` , `level` , `xpNext` .

`usecases/saveCampionamentoUseCase.js`

US-016 — Storico campionamenti

Come trainer **voglio** visualizzare lo storico dei campionamenti precedenti **per** monitorare il progresso nel tempo.

Lista cronologica dei campionamenti passati con data, valori per ogni stat e delta rispetto al campionamento precedente evidenziato in verde (miglioramento) o rosso (peggioramento). La sezione è collapsabile per non occupare spazio nella tab.

`features/client/ClientDashboard.jsx`

US-017 — Note thread trainer → cliente

Come trainer **voglio** aggiungere note testuali e rispondere ai commenti del cliente **per** comunicare in modo strutturato.

Le note sono thread a due livelli: il trainer crea la nota root (`parentId: null`), il cliente risponde con commenti (`parentId: noteId`). Il trainer può aggiungere sia note root che commenti su qualsiasi thread. L'autore o un trainer/org_admin può eliminare ogni nota.

`features/client/client-dashboard/NotesSection.jsx`

US-018 — Elimina nota o thread a cascata

Come trainer **voglio** eliminare una nota o un thread a cascata **per** rimuovere contenuti non pertinenti.

Eliminare una nota root elimina automaticamente tutti i commenti figli (gestito lato app, non via regole Firestore). Solo l'autore o un trainer/org_admin possono eliminare. L'azione è preceduta da conferma per prevenire cancellazioni accidentali.

`features/client/client-dashboard/NotesSection.jsx`

US-019 — Crea scheda allenamento multi-giorno

Come trainer **voglio** creare e assegnare una scheda allenamento multi-giorno **per** strutturare il piano di lavoro del cliente.

`WorkoutPlanForm` permette di definire fino a 7 giorni, ognuno con una lista di esercizi (nome, serie, ripetizioni, recupero in secondi, note). La scheda viene salvata in `organizations/{orgId}/workoutPlans/{planId}` con `status: 'active'` e `clientId` associato.

`features/client/client-dashboard/WorkoutPlanSection.jsx`

US-020 — Archivia scheda e crea nuova

Come trainer **voglio** archiviare una scheda e crearne una nuova **per** mantenere uno storico degli allenamenti.

Creare una nuova scheda archivia automaticamente quella corrente (`status: 'archived'`). Lo storico delle schede archiviate è visibile in un accordion collapsabile sotto la scheda attiva, con titolo, data e possibilità di eliminazione.

`features/client/client-dashboard/WorkoutPlanSection.jsx`

US-021 — Misure antropometriche nel tempo

Come trainer **voglio** registrare il peso e l'altezza del cliente nel tempo **per** monitorare le misure antropometriche.

`MisureSection` mostra una timeline storica di peso e altezza con trend inline (freccia su/giù e delta numerico rispetto alla misura precedente). I dati sono salvati in `clients/{clientId}.misure[]` tramite `useMisure`. Ogni entry include data e valori.

`features/client/client-dashboard/MisureSection.jsx`

US-022 — Esporta PDF cliente con scelta tema

Come trainer **voglio** esportare un report PDF del cliente scegliendo il tema (dark o B&W) **per** consegnare una documentazione adatta a stampa o visualizzazione digitale.

`PrintPickerModal` chiede prima della stampa se usare il tema scuro (per visualizzazione digitale) o bianco/nero (per stampa fisica). `ClientReportPrint` inietta CSS `@media print` in `document.head` e chiama `window.print()`. Il report include: anagrafica, test con delta rispetto al precedente, BIA se presente, ultimi 5 campionamenti. Nessuna dipendenza esterna: il browser genera il PDF nativamente.

`features/client/client-dashboard/ClientReportPrint.jsx`, `components/common/PrintPickerModal.jsx`

US-082 — Calendario sessioni cliente (tab in dashboard trainer)

Come trainer **voglio** visualizzare il calendario sessioni del cliente direttamente nella sua dashboard **per** controllare frequenza e storico senza cambiare pagina.

La tab "Sessioni" della dashboard trainer incorpora `ClientCalendar` filtrato per il cliente selezionato. Mostra slot passati (completati / saltati) e futuri (pianificati), permettendo al trainer di verificare la frequenza di partecipazione senza uscire dalla scheda cliente.

`features/client/client-dashboard/ClientSessionsSummary.jsx`

US-083 — Grafico andamento percentili nel tempo

Come trainer **voglio** visualizzare il grafico di andamento dei percentili filtrabile per singola stat **per** analizzare l'evoluzione di ogni capacità atletica campionato per campionato.

`StatsChart` mostra un `LineChart` (Recharts) con i campionamenti sull'asse X e il percentile (0–100) sull'asse Y. I bottoni di selezione filtrano per singola stat (velocità, forza, resistenza, ecc.). Il componente è visibile solo quando il cliente ha almeno 2 campionamenti.

`features/client/StatsChart.jsx`

Test Atletici

US-023 — Guida dettagliata dei test

Come trainer **voglio** vedere la guida dettagliata di ogni test **per** eseguire le misurazioni in modo corretto e standardizzato.

`TestGuidePage` elenca i test del modulo attivo come card espandibili, ognuna con protocollo di esecuzione, attrezzatura richiesta, unità di misura e note tecniche. In Soccer Academy mostra solo i test del protocollo soccer, escludendo quelli PT.

`features/trainer/TestGuidePage.jsx`

US-024 — Banner età fuori fascia normativa

Come trainer **voglio** sapere quando l'età del cliente è fuori dal range normativo **per** interpretare correttamente i percentili stimati.

`getAgeGroupClamped` in `utils/tables.js` restituisce `outOfRange: true` quando l'età è fuori dalla fascia per cui esiste una tabella normativa. In `CampionamentoView` questo genera un banner ambra sopra il campo del test interessato, spiegando che il percentile è una stima basata sulla fascia più vicina disponibile.

`features/client/CampionamentoView.jsx`

US-025 — Filtraggio test per modulo attivo

Come trainer **voglio** vedere solo i test disponibili per il modulo attivo **per** non confondere test di domini diversi.

In PT i test sono filtrati per la categoria del cliente (health / active / athlete). In Soccer Academy vengono mostrati solo i test della fascia d'età del cliente (Pulcini / Esordienti / Senior). La fonte di verità è `getTestsForCategoria` in `constants/index.js`, alimentata da `SOCCER_FIXED_TESTS` e dalla categoria PT.

`features/trainer/TestGuidePage.jsx`

BIA — Bioimpedenziometria

US-026 — Inserimento parametri BIA

Come trainer **voglio** inserire i parametri BIA del cliente **per** monitorare la composizione corporea.

Il form raccoglie 8 parametri: massa grassa %, massa muscolare kg, acqua %, massa ossea kg, BMI (calcolato automaticamente da peso/altezza), metabolismo basale kcal, età metabolica, grasso viscerale. Il BMI viene ricalcolato automaticamente ogni volta che peso o altezza cambiano.

`features/bia/BiaView.jsx`

US-027 — Indicatori cromatici per range clinici BIA

Come trainer **voglio** vedere i parametri BIA con indicatori cromatici **per** identificare rapidamente i valori fuori norma.

`BiaGaugeBar` colora ogni barra in base ai range clinici definiti in `constants/bia.js`, tenendo conto del sesso del cliente: verde = ottimale, giallo = attenzione, rosso = critico. Per i parametri con direzione `inverse` (es. massa grassa: meno è meglio) la scala cromatica è invertita.

`features/bia/bia-view/BiaGaugeBar.jsx`

US-028 — Storico BIA con grafico

Come trainer **voglio** visualizzare lo storico BIA con grafico **per** seguire l'evoluzione della composizione corporea nel tempo.

`BiaHistoryChart` mostra un `LineChart` per i parametri chiave (massa grassa, muscolare, acqua) sull'asse del tempo. I dati provengono da `client.biaHistory[]`. L'utente può selezionare quale parametro visualizzare tramite bottoni di filtro.

`features/bia/bia-view/BiaHistoryChart.jsx`

US-029 — BIA non disponibile in Soccer Academy

Come trainer **voglio** essere informato che la BIA non è disponibile per il modulo Soccer **per** evitare operazioni su profili non supportati.

`BiaLockedPanel` viene mostrato al posto della tab BIA quando il modulo dell'organizzazione è `soccer_academy`. Il pannello spiega che la bioimpedenziometria non è supportata per questo modulo e che i profili soccer sono sempre `tests_only`.

```
features/bia/BiaLockedPanel.jsx
```

US-084 — Riepilogo composizione corporea inline (BiaSummary)

Come trainer **voglio** vedere il riepilogo della composizione corporea con gauge bar e rank BIA affiancati **per** avere un colpo d'occhio sullo stato fisico senza aprire la tab dedicata.

`BiaSummary` mostra tutti i parametri BIA non calcolati con le relative barre gauge cromatiche, affiancati dal rank BIA (calcolato separatamente rispetto al rank atletico dei test). Appare nella tab Panoramica della dashboard quando il cliente ha almeno una misurazione BIA salvata.

```
features/bia/bia-view/BiaSummary.jsx
```

Sessioni e Calendario

US-030 — Calendario trainer (mese / settimana / giorno)

Come trainer **voglio** visualizzare il calendario delle sessioni in vista mese, settimana o giorno **per** avere una visione completa degli impegni.

`TrainerCalendar` mostra gli slot sessione con clienti e gruppi assegnati. La vista default è settimana. Cliccando su uno slot si apre `SlotPopup` con le azioni disponibili (chiudi / salta / modifica). I colori differenziano gli stati: pianificato (ciano), completato (verde), saltato (grigio).

```
features/trainer/TrainerCalendar.jsx
```

US-031 — Crea slot sessione

Come trainer **voglio** creare uno slot sessione con clienti e/o gruppi assegnati **per** pianificare un allenamento.

`AddSlotModal` raccoglie data, ora inizio/fine, clienti singoli e/o gruppi partecipanti. Lo slot viene salvato con `status: 'planned'`. I clienti appartenenti ai gruppi assegnati vengono inclusi automaticamente in `clientIds`.

```
features/trainer/trainer-calendar/AddSlotModal.jsx
```

US-032 — Crea ricorrenza (sessioni ripetute)

Come trainer **voglio** creare una ricorrenza su giorni della settimana **per** pianificare allenamenti ricorrenti senza inserirli uno per uno.

`RecurrenceModal` permette di scegliere i giorni della settimana, il periodo (data inizio/fine) e l'orario. Al salvataggio genera automaticamente tutti gli slot per l'intero periodo. La ricorrenza è un'entità di primo livello con `status: 'active' | 'ended' | 'cancelled'`.

```
features/trainer/trainer-calendar/RecurrenceModal.jsx
```

US-033 — Chiudi sessione (segna presenti e assenti)

Come trainer **voglio** chiudere una sessione segnando presenti e assenti **per** assegnare XP agli atleti presenti e notificare gli assenti.

`CloseSessionModal` mostra la lista dei clienti dello slot. Il trainer spunta i presenti. Alla conferma: i presenti ricevono XP con moltiplicatore streak, gli assenti ricevono una notifica automatica. Lo slot passa a `status: 'completed'` e i dati vengono salvati in `attendees` e `absentees`.

`features/trainer/trainer-calendar/CloseSessionModal.jsx`

US-034 — Salta sessione

Come trainer **voglio** saltare una sessione senza assegnare XP **per** registrare un appuntamento cancellato.

Dalla `SlotPopup`, l'azione "Salta" imposta `status: 'skipped'`. Non genera XP né notifiche. Lo slot rimane visibile nel calendario con styling grigio per mantenere la tracciabilità degli appuntamenti cancellati.

`features/trainer/trainer-calendar/SlotPopup.jsx`

US-035 — Gestione lista ricorrenze

Come trainer **voglio** gestire le ricorrenze (estendi periodo, modifica orario, annulla, elimina slot futuri) **per** mantenere il calendario aggiornato.

`RecurrencesPage` mostra le ricorrenze attive per default e un accordion archivio per quelle `ended` o `cancelled`. La lista attive è paginata (10 per pagina). Ogni card riporta giorni della settimana, orario, numero clienti assegnati e settimane rimanenti.

`features/trainer/RecurrencesPage.jsx`

US-092 — Dettaglio singola ricorrenza

Come trainer **voglio** aprire il dettaglio di una ricorrenza specifica **per** verificarne e correggerne i dati senza passare dal calendario.

`RecurrenceDetailView` mostra l'intestazione con giorni, orario e data di fine, un layout a due colonne con i clienti assegnati (con ricerca testuale) e il contatore totale di settimane calcolate. Permette di annullare la ricorrenza con conferma, eliminando tutti gli slot futuri associati.

`features/trainer/recurrences-page/RecurrenceDetailView.jsx`

Gruppi

US-036 — Crea e gestisci gruppi

Come trainer **voglio** creare e gestire gruppi di clienti **per** organizzare gli atleti per squadra o livello.

`GroupsPage` lista i gruppi dell'organizzazione con nome e numero di clienti. Supporta creazione, rinomina ed eliminazione. Un gruppo è una struttura in `organizations/{orgId}/groups/{groupId}` contenente un array di `clientIds`.

`features/trainer/GroupsPage.jsx`

US-037 — Aggiungi/rimuovi cliente da gruppo con preview calendario

Come trainer **voglio** aggiungere o rimuovere un cliente da un gruppo con conferma e preview dell'impatto sul calendario **per** capire quanti slot saranno aggiornati prima di confermare.

`GroupToggleDialog` mostra in anteprima quanti slot futuri non ricorrenti e quante ricorrenze attive verranno aggiornati. Alla conferma, il sistema aggiorna `group.clientIds`, sincronizza gli slot futuri non ricorrenti e le ricorrenze attive con i loro slot futuri. Gli slot passati non vengono mai modificati.

`features/trainer/groups-page/GroupToggleDialog.jsx`

US-038 — Classifica del gruppo

Come trainer **voglio** vedere la classifica del gruppo ordinata per media o per singola stat **per** confrontare le performance degli atleti.

`GroupLeaderboard` mostra tutti i clienti del gruppo ordinati per media percentile (default) o per una singola stat selezionabile. I primi 3 atleti sono visualizzati con un podio visivo. La lista è paginata. Disponibile nel tab "Classifica" di `GroupDetailView`.

`features/trainer/groups-page/GroupLeaderboard.jsx`

US-039 — Campioni per disciplina (griglia)

Come trainer **voglio** vedere i campioni per disciplina del gruppo **per** identificare i migliori atleti per ogni test.

`GroupChampions` mostra una griglia con un riquadro per ogni stat/test. In ogni riquadro compare il nome del cliente con il percentile più alto per quella disciplina. Disponibile nel tab "Analisi" di `GroupDetailView`.

`features/trainer/groups-page/GroupChampions.jsx`

US-040 — Analisi aggregata del gruppo

Come trainer **voglio** vedere l'analisi aggregata del gruppo: riepilogo, trend, heatmap, più migliorati **per** avere insights complessivi sulle performance.

`GroupAnalysis` include: riepilogo delle statistiche medie del gruppo, LineChart del trend nel tempo per stat selezionabile, heatmap delle presenze per cliente/mese, e lista dei clienti più migliorati (delta percentile massimo tra due campionamenti consecutivi). Disponibile nel tab "Analisi".

`features/trainer/groups-page/GroupAnalysis.jsx`

US-041 — Confronto visivo fino a 3 atleti (radar + tabella)

Come trainer **voglio** confrontare visivamente fino a 3 atleti dello stesso gruppo **per** fare analisi comparative dettagliate.

`GroupComparison` ha un selettore paginato per scegliere fino a 3 atleti del gruppo. Il radar SVG multi-overlay sovrappone i profili in un unico grafico pentagono. La tabella sotto riporta i valori numerici per ogni stat, con evidenziazione del valore più alto per ogni riga. Disponibile nel tab "Confronto".

`features/trainer/groups-page/GroupComparison.jsx`

US-085 — Sessioni del gruppo con statistiche presenze

Come trainer **voglio** vedere le sessioni del gruppo (prossime e recenti) con statistiche aggregate **per** monitorare la frequenza senza aprire il calendario.

`GroupSessionsPanel` mostra tre metriche aggregate: sessioni completate negli ultimi 30 giorni, sessioni pianificate, tasso medio di presenze. Sotto, due liste paginate: slot futuri pianificati e slot recenti completati con rapporto presenti/invitati per ogni slot. Disponibile nel tab "Sessioni" di `GroupDetailView`.

`features/trainer/groups-page/GroupSessionsPanel.jsx`

US-086 — Note/annunci di gruppo

Come trainer **voglio** pubblicare note e annunci di gruppo e cancellarli **per** comunicare informazioni a tutto il gruppo in modo strutturato.

`GroupNotes` è distinta dalle note individuali dei clienti. Ogni nota è un annuncio flat (senza thread nidificati) visibile a tutti i trainer dell'organizzazione. Solo l'autore o un `org_admin` possono eliminarla. Il trainer può inviare premendo `Ctrl+Enter`. La lista è paginata (8 per pagina). Disponibile nel tab "Note" di `GroupDetailView`.

`features/trainer/groups-page/GroupNotes.jsx`

US-087 — Esporta PDF gruppo con scelta tema

Come trainer **voglio** esportare un report PDF del gruppo scegliendo il tema (dark o B&W) **per** produrre una documentazione stampabile con classifica, analisi e confronto atleti.

Come per il PDF cliente, `PrintPickerModal` chiede prima la scelta del tema. `GroupReportPrint` usa `window.print()` con CSS `@media print` iniettato nel DOM. Il report include: classifica finale del gruppo, campioni per disciplina e riepilogo statistiche aggregate. Nessuna dipendenza esterna.

`features/trainer/groups-page/GroupReportPrint.jsx`, `components/common/PrintPickerModal.jsx`

Wearable / Attività Fisica

US-088 — Trainer abilita wearable e visualizza dati attività cliente

Come trainer **voglio** abilitare il tracciamento wearable per un cliente e visualizzare passi, calorie e livello di attività degli ultimi 7 giorni **per** monitorare l'attività quotidiana al di fuori delle sessioni.

`WearableSection` nella dashboard trainer mostra un indicatore colorato del livello di attività (basso/medio/alto basato sui passi medi degli ultimi 7 giorni), tre ring con passi, calorie e minuti attivi del giorno corrente, e un `BarChart` con i passi giornalieri degli ultimi 7 giorni. Il trainer abilita la funzione per il cliente; il collegamento effettivo avviene lato client tramite OAuth Google Fit.

`features/client/client-dashboard/WearableSection.jsx`, `hooks/useWearable.js`

US-089 — Client collega il proprio account Google Fit

Come client **voglio** collegare il mio account Google Fit all'app **per** condividere i dati di attività con il trainer.

`ClientWearableSection` nella dashboard client mostra un pulsante per avviare il flusso OAuth Google Fit via `linkWithRedirect`. Dopo il redirect, `resolveGoogleFitRedirect` recupera l'access token, legge i dati di attività tramite Google Fit REST API e li salva in Firestore. Il trainer vede i dati aggiornati nella propria `WearableSection`.

`features/client/client-view/ClientWearableSection.jsx`

Gamification

US-042 — Livello, XP e progressione

Come trainer **voglio** vedere il livello, gli XP e gli XP necessari per il prossimo livello del cliente **per** monitorare la progressione gamificata.

Il livello avanza quando $xp \geq xpNext$. La soglia del livello successivo è calcolata come $xpNext \times 1.08$ (curva raggiungibile in ~3,5–4 anni a 3 sessioni/settimana). La XPBar mostra la percentuale di completamento verso il prossimo livello. Il livello iniziale è 1 con $xpNext = 500$.

```
features/client/ClientDashboard.jsx
```

US-043 — Rank atletico

Come trainer **voglio** vedere il rank atletico del cliente **per** avere una valutazione sintetica e motivante del livello.

`getRankFromMedia(media)` restituisce il rank in base alla media percentile: IRON → BRONZE → SILVER → GOLD → PLATINUM → DIAMOND. Rank e colore vengono aggiornati ad ogni campionamento salvato. La BIA non influenza il rank: è determinato esclusivamente dai test atletici.

```
utils/gamification.js
```

US-044 — Grafico XP nel tempo

Come trainer **voglio** vedere il grafico XP per giorno, settimana o mese **per** analizzare la costanza e l'evoluzione della partecipazione.

`XPTrendChart` legge `client.log[]` dove ogni entry contiene `ts: Date.now()` e la quantità di XP guadagnata. I tre bottoni di granularità aggregano i dati sull'asse temporale scelto. Il grafico è disponibile sia nella dashboard trainer che in quella client.

```
features/client/client-dashboard/XPTrendChart.jsx
```

US-045 — XP bonus per streak di sessioni consecutive

Come trainer **voglio** che il cliente riceva XP bonus per sessioni consecutive **per** incentivare la partecipazione regolare.

`calcSessionXP(baseXP, streak)` applica il moltiplicatore $1 + \min(streak \times 0.1, 1.0)$. A streak 10 il moltiplicatore raggiunge il massimo di $\times 2.0$. La streak si incrementa ad ogni sessione chiusa con presenza del cliente, e si azzerà in caso di assenza.

```
utils/gamification.js
```

US-046 — XP da BIA e campionamenti in base ai miglioramenti

Come trainer **voglio** che il cliente riceva XP da BIA e campionamenti in base ai miglioramenti effettivi **per** premiare i progressi in tutte le dimensioni del profilo.

Campionamento — tier basato sul numero di stat percentili migliorate: FIRST=50 (primo campionamento), NONE=10, PARTIAL=30 (1 stat), MOST=60 (2–3 stat), ALL=100 (4+ stat). **BIA** — tier basato su 4 parametri chiave migliorati (massa grassa_l, massa muscolare_l, acqua_l, grasso viscerale_l): stessa scala di XP.

```
usecases/saveBiaUseCase.js , usecases/saveCampionamentoUseCase.js
```

Notifiche

US-047 — Invia notifica al cliente

Come trainer **voglio** inviare notifiche al cliente **per** comunicare feedback sulle presenze.

Le notifiche vengono create automaticamente alla chiusura di una sessione per i clienti assenti. Il trainer può anche inviarle manualmente. I dati sono persistiti in `organizations/{orgId}/notifications/{notId}` con tipo, messaggio, data e flag `read`.

```
firebase/services/notifications.js
```

US-048 — Pannello notifiche (client)

Come client **voglio** vedere le notifiche ricevute in un pannello dedicato **per** essere informato delle comunicazioni del trainer.

`NotificationsPanel` mostra le notifiche ordinate per data, con un badge contatore per le non lette. Ogni notifica riporta tipo, messaggio e data. Cliccando una singola notifica la marca come letta aggiornando `read: true` e `readAt`.

```
features/notification/NotificationsPanel.jsx
```

US-049 — Segna tutte le notifiche come lette

Come client **voglio** marcare tutte le notifiche come lette in un colpo solo **per** gestire rapidamente il pannello.

`useNotifications` espone `markAllRead()` che aggiorna in batch tutti i documenti notifica con `read: true` e `readAt: now()`. Il badge contatore scompare immediatamente tramite optimistic update, prima che la write Firestore sia completata.

```
hooks/useNotifications.js
```

Area Self-Service Cliente

US-050 — Dashboard cliente (vista client)

Come client **voglio** vedere la mia dashboard con avatar, rank, XP e statistiche **per** monitorare i miei progressi atletici.

`ClientDashboardPage` usa lo stesso layout della vista trainer (2 colonne su desktop, tab AVATAR su mobile) ma in sola lettura: il client non può aprire il campionamento né modificare dati. Vede rank, XP, statistiche percentili, storico campionamenti e tutte le tab del proprio profilo.

```
features/client/client-view/ClientDashboardPage.jsx
```

US-051 — Scheda allenamento attiva (read-only, client)

Come client **voglio** vedere la scheda allenamento attiva assegnata dal trainer **per** seguire il piano di allenamento indicato.

`ClientWorkoutSection` carica la scheda attiva con `getWorkoutPlanForClient` (query filtrata per `clientId` e `status: 'active'`). Mostra i giorni come tab (max 7) con la lista degli esercizi per ogni giorno: nome, serie × ripetizioni, recupero in secondi e note del trainer. Sola lettura.

```
features/client/client-dashboard/ClientWorkoutSection.jsx
```

US-052 — Calendario sessioni (client)

Come client **voglio** vedere il calendario delle mie sessioni **per** sapere quando ho allenamenti in programma.

`ClientCalendar` filtra gli slot per `clientId` contenente l'ID del cliente autenticato. Mostra slot pianificati, completati (con indicazione presenza o assenza) e saltati. Il client vede solo i propri slot, non quelli degli altri clienti dell'organizzazione.

`features/client/ClientCalendar.jsx`

US-053 — Commenta le note del trainer

Come client **voglio** commentare le note del trainer sul mio profilo **per** partecipare al dialogo.

Il client può aggiungere solo commenti (`parentId != null`) su thread esistenti creati dal trainer. Non può creare note root. Firestore rules verificano che `parentId` non sia null per la scrittura del ruolo client. I suoi commenti appaiono nel thread con il badge ruolo "Client".

`features/client/client-dashboard/NotesSection.jsx`

US-054 — Notifiche client

Come client **voglio** vedere le mie notifiche e marcarle come lette **per** gestire le comunicazioni ricevute.

Stessa `NotificationsPanel` degli altri ruoli, filtrata per il `clientId` dell'utente autenticato. Il badge nell'header mostra il numero di notifiche non lette.

`features/notification/NotificationsPanel.jsx`

US-055 — Cambio password (area profilo client)

Come client **voglio** modificare la mia password dall'area profilo **per** mantenere la sicurezza dell'account.

Diverso da `ChangePasswordScreen` (obbligatorio al primo accesso): qui il client modifica volontariamente la password. La funzione si trova in `ClientProfilePage` e richiede la password corrente per re-auth prima di consentire l'aggiornamento.

`features/client/client-view/ClientProfilePage.jsx`

US-056 — Trend XP nel tempo (client)

Come client **voglio** vedere il mio trend XP nel tempo **per** seguire la mia evoluzione gamificata.

Stessa `XPTrendChart` disponibile nella dashboard trainer, con granularità giorno/settimana/mese. Il client vede esclusivamente i propri dati. Disponibile come tab dedicata nella propria dashboard.

`features/client/client-dashboard/XPTrendChart.jsx`

Profilo Trainer

US-057 — Modifica email e password (trainer)

Come trainer **voglio** modificare la mia email e la mia password **per** mantenere le credenziali aggiornate e sicure.

`ProfilePage` gestisce due form separati. Il cambio email usa `verifyBeforeUpdateEmail` : Firebase invia un link di verifica al nuovo indirizzo prima che il cambio diventi effettivo. Il cambio password usa re-auth con la password corrente. Entrambe le azioni generano audit log con `AUDIT_ACTIONS.EMAIL_CHANGED` e `AUDIT_ACTIONS.PASSWORD_CHANGED` .

features/trainer/ProfilePage.jsx

Area Org Admin

US-058 — Dashboard organizzazione

Come org_admin **voglio** vedere la dashboard dell'organizzazione con statistiche aggregate **per** avere una visione d'insieme dell'attività.

OrgDashboard mostra contatori di clienti attivi, sessioni del mese e trainer, un riepilogo del piano con utilizzo corrente (basato su memberCount e clientCount atomici) e un feed di attività recente. Punto di ingresso dell'area org_admin.

features/org/org-pages/OrgDashboard.jsx

US-059 — Lista e gestione membri del team

Come org_admin **voglio** visualizzare e gestire i membri del team **per** sapere chi ha accesso all'organizzazione.

MembersPage mostra tutti i membri con ruolo, email e data di ingresso. Visualizza un banner giallo e disabilita il bottone AGGIUNGI quando memberCount >= getPlanLimits(plan).trainers , impedendo di superare il limite del piano.

features/org/org-pages/MembersPage.jsx

US-060 — Aggiungi membro al team

Come org_admin **voglio** aggiungere nuovi membri assegnando il ruolo **per** espandere il team con le autorizzazioni corrette.

CreateMemberForm crea l'account Firebase Auth tramite un'app secondaria (per non effettuare il logout dall'account corrente), crea il documento /users/{uid} e aggiunge il membro con addMember — un batch che scrive il documento membro e incrementa atomicamente memberCount sull'organizzazione.

features/org/org-pages/CreateMemberForm.jsx

US-061 — Rimuovi membro del team

Come org_admin **voglio** rimuovere un membro con conferma **per** revocare l'accesso a chi non fa più parte dell'org.

removeMember elimina il documento /members/{uid} , aggiorna il profilo in /users/{uid} e decrementa memberCount atomicamente in un batch. L'azione è registrata nell'audit log con AUDIT_ACTIONS.MEMBER_REMOVED .

features/org/org-pages/MembersPage.jsx

US-062 — Cambia ruolo membro

Come org_admin **voglio** cambiare il ruolo di un membro **per** adeguare i permessi alle responsabilità attuali.

La select aggiorna sia /members/{uid} che /users/{uid} . Le Firestore rules verificano via isOrgAdminForMember che il nuovo ruolo sia solo org_admin , trainer o staff_readonly , impedendo escalation al ruolo super_admin . L'azione è tracciata con AUDIT_ACTIONS.ROLE_CHANGED .

features/org/org-pages/MembersPage.jsx

Area Super Admin

US-063 — Dashboard globale

Come super_admin **voglio** vedere la dashboard globale con statistiche su tutte le organizzazioni **per** monitorare l'utilizzo della piattaforma.

AdminDashboard aggrega contatori totali (organizzazioni attive, trainer, clienti), un breakdown per piano (free / pro / enterprise) e la lista delle organizzazioni create più di recente. L'intera area admin ha accent rosso per differenziarla visivamente dall'area trainer.

```
features/admin/admin-pages/AdminDashboard.jsx
```

US-064 — Lista tutte le organizzazioni

Come super_admin **voglio** vedere la lista di tutte le organizzazioni con utilizzo del piano **per** fare customer service e monitorare i limiti.

OrgsPage mostra tutte le organizzazioni con nome, piano, data di creazione e contatori trainer/clienti. Cliccando un'organizzazione si accede al suo dettaglio. Supporta ricerca testuale per nome organizzazione.

```
features/admin/admin-pages/OrgsPage.jsx
```

US-065 — Crea nuova organizzazione

Come super_admin **voglio** creare una nuova organizzazione **per** onboardare nuovi clienti B2B.

CreateOrgForm genera un orgId da slug del nome + suffisso random per garantire unicità. Il campo ownerId è prepopolato con l'uid del super_admin autenticato. Imposta il piano iniziale e crea il documento /organizations/{orgId} con i counter azzerati.

```
features/admin/admin-pages/CreateOrgForm.jsx
```

US-066 — Dettaglio organizzazione con utilizzo piano

Come super_admin **voglio** vedere il dettaglio di un'organizzazione con barre di utilizzo trainer/clienti **per** valutare quando si avvicina al limite.

OrgDetailView mostra barre progresso per trainer (memberCount / limit) e clienti (clientCount / limit). I colori virano al giallo/rosso avvicinandosi al limite. Mostra anche la lista dei membri correnti con possibilità di rimozione di emergenza.

```
features/admin/admin-pages/OrgDetailView.jsx
```

US-067 — Rimuovi membro da qualsiasi organizzazione

Come super_admin **voglio** rimuovere un membro da qualsiasi organizzazione **per** gestire situazioni di emergenza o richieste di supporto.

Stesso flusso di rimozione dell'org_admin, ma accessibile globalmente dal super_admin senza essere membro dell'organizzazione target. Utile per gestire account compromessi o richieste urgenti di disattivazione da parte dei clienti B2B.

```
features/admin/admin-pages/OrgDetailView.jsx
```

US-090 — Profilo super_admin (email e password)

Come super_admin **voglio** modificare la mia email e la mia password dall'area profilo **per** mantenere le credenziali dell'account amministratore aggiornate e sicure.

`AdminProfilePage` replica la funzionalità di `ProfilePage` (trainer) nell'area admin con accent rosso. Gestisce separatamente il cambio email (con `verifyBeforeUpdateEmail`, link di verifica prima del cambio effettivo) e il cambio password (con re-auth). Entrambe le azioni generano audit log.

`features/admin/admin-pages/AdminProfilePage.jsx`

Staff Read-Only

US-068 — Accesso in sola lettura

Come staff_readonly **voglio** accedere alla piattaforma in sola lettura senza poter modificare nulla **per** consultare i dati operativi senza rischio di alterarli.

`ReadonlyContext` espone il booleano `readonly`. `ReadonlyGuard` avvolge tutti i controlli di modifica (bottoni aggiungi / salva / elimina) e li nasconde quando `readonly=true`. `ReadonlyBanner` mostra un banner informativo fisso in cima alla pagina. Il routing è identico al ruolo trainer.

`components/common/ReadonlyGuard.jsx`, `context/ReadonlyContext.jsx`

Guida Test

US-069 — Guida di esecuzione dei test

Come trainer **voglio** consultare la guida di esecuzione di ogni test **per** standardizzare le misurazioni e formare il personale.

Ogni test in `constants/tests.js` ha un campo `guide` con il protocollo testuale. `TestGuidePage` li visualizza in card espandibili con attrezzatura necessaria, procedura passo-passo, unità di misura e note tecniche. In Soccer Academy mostra solo i test del protocollo soccer, escludendo quelli PT.

`features/trainer/TestGuidePage.jsx`

Modulo Soccer Academy

US-070 — Crea allievo con ruolo calcistico

Come coach (soccer) **voglio** creare un allievo con ruolo calcistico **per** classificare gli atleti per posizione in campo.

Lo step ruolo del wizard soccer usa `PLAYER_ROLES` da `config/modules.config.js` (portiere / difensore / centrocampista / attaccante). Il ruolo è esclusivamente un'etichetta visiva: non determina i test somministrati (determinati dalla fascia d'età), ma appare come badge colorato nella lista clienti e nella scheda atleta.

`components/modals/new-client-wizard/steps/StepRuolo.jsx`

US-071 — Fascia d'età automatica da data di nascita

Come coach (soccer) **voglio** che la fascia d'età dell'allievo sia derivata automaticamente dall'età **per** non dover classificare manualmente ogni atleta.

`getCategoriaFromEta(eta)` in `config/modules.config.js` calcola: età < 10 → `soccer_youth` (Pulcini), 10–13 → `soccer_junior` (Esordienti), 14+ → `soccer` (Senior). La categoria viene ricalcolata ad ogni aggiornamento dell'età e salvata nel campo `categoria` del cliente, che guida il filtraggio dei test.

`config/modules.config.js`

US-072 — Test specifici per fascia d'età soccer

Come coach (soccer) **voglio** vedere solo i test specifici per la fascia d'età dell'allievo **per** effettuare campionamenti pertinenti al protocollo soccer.

`getTestsForCategoria(categoria)` filtra `ALL_TESTS` per la fascia. Pulcini (7–9): 5 test motori base. Esordienti (10–13): 5 test coordinazione/resistenza. Senior (14+): 5 test di performance avanzata. Il `standing_long_jump` è l'unico test condiviso tra tutte e tre le fasce.

`constants/index.js`

US-073 — Filtra allievi per ruolo e fascia d'età

Come coach (soccer) **voglio** filtrare gli allievi per ruolo e fascia d'età **per** gestire categorie numerose in modo efficiente.

`FiltersSidebar` in modalità soccer mostra due gruppi di filtri: per ruolo calcistico e per fascia (Pulcini / Esordienti / Senior). Il filtro FASCIA appare solo quando sono presenti clienti appartenenti ad almeno due fasce diverse nell'organizzazione, evitando filtri inutili per org con una sola fascia.

`features/trainer/clients-page/FiltersSidebar.jsx`

Upgrade Profilo Cliente

US-074 — Upgrade tests_only → complete

Come trainer **voglio** aggiornare il profilo di un cliente da `tests_only` a `complete` **per** aggiungere il modulo BIA mantenendo lo storico test atletico.

`upgradeProfileUseCase` imposta `profileType: 'complete'`, conserva `stats`, `campionamenti`, `log`, `rank`, `media` e azzerà `biaHistory: []` e `lastBia: null`. Il cliente partirà da zero con la BIA, ma mantiene tutta la progressione atletica accumulata.

`usecases/upgradeProfileUseCase.js`

US-075 — Upgrade bia_only → complete

Come trainer **voglio** aggiornare il profilo di un cliente da `bia_only` a `complete` **per** aggiungere i test atletici mantenendo lo storico BIA.

`upgradeProfileUseCase` imposta `profileType: 'complete'`, conserva `biaHistory` e `lastBia`, azzerà `stats: {}` e `campionamenti: []`. Il cliente partirà da zero con i test atletici, ma mantiene tutta la storia della composizione corporea.

`usecases/upgradeProfileUseCase.js`

US-091 — Banner suggerimento upgrade profilo parziale

Come trainer **voglio** vedere un banner di suggerimento quando il profilo del cliente è parziale **per** essere informato dell'opportunità di upgrade senza cercare la funzione nel menu.

`UpgradeCategoryBanner` appare nella dashboard quando `profileType !== 'complete'`. Per `tests_only` suggerisce di aggiungere la BIA; per `bia_only` suggerisce di aggiungere i test. Include un `ConfirmDialog` che esplicita le conseguenze: quali dati vengono mantenuti e quali azzerati, prima di procedere all'upgrade.

`features/bia/UpgradeCategoryBanner.jsx`

Wizard Nuovo Cliente

US-076 — Step anagrafica

Come trainer **voglio** inserire i dati anagrafici del cliente nel primo step del wizard **per** raccogliere le informazioni di base prima di procedere.

`StepAccount` raccoglie nome, data di nascita, sesso, peso, altezza ed email. La data di nascita viene usata per calcolare l'età che, in Soccer Academy, determina automaticamente la fascia d'età. Tutti i campi sono validati prima di permettere il passaggio allo step successivo.

`components/modals/new-client-wizard/steps/StepAccount.jsx`

US-077 — Step categoria (PT) o ruolo (Soccer)

Come trainer **voglio** selezionare la categoria PT o il ruolo soccer nel secondo step del wizard **per** assegnare il profilo corretto al tipo di cliente.

In PT questo step è `StepCategoria` (health / active / athlete). In Soccer Academy è `StepRuolo` (portiere / difensore / centrocampista / attaccante). La selezione è differenziata in base al `moduleType` dell'organizzazione. Il wizard Soccer ha 3 step fissi; quello PT ne ha fino a 4 o 5 in base al profilo selezionato.

`components/modals/new-client-wizard/steps/StepRuolo.jsx`

US-078 — Step credenziali (ultimo step wizard)

Come trainer **voglio** creare le credenziali di accesso del cliente nell'ultimo step **per** permettere al cliente di accedere alla propria area.

Al submit, `useWizard` crea l'utente Firebase Auth tramite app secondaria, imposta `mustChangePassword: true`, salva il documento `/users/{uid}`, crea il cliente in Firestore e registra `AUDIT_ACTIONS.CLIENT_CREATED`. Se uno dei passaggi fallisce, viene tentato il rollback per evitare stati inconsistenti.

`components/modals/new-client-wizard/useWizard.js`

US-093 — Step selezione categoria PT (StepCategoria)

Come trainer **voglio** selezionare la categoria PT del cliente (health / active / athlete) con la descrizione dei test associati **per** assegnare il protocollo di valutazione corretto fin dalla creazione.

`StepCategoria` presenta le 3 categorie come card cliccabili con nome, colore distintivo e lista dei 5 test che verranno somministrati. La selezione determina i test visualizzati in tutti i campionamenti futuri del cliente. Questo step è presente solo nel wizard PT e assente nel wizard Soccer, dove la categoria è sempre derivata dall'età.

`components/modals/new-client-wizard/steps/StepCategoria.jsx`

US-094 — Step tipo profilo (StepProfileType)

Come trainer **voglio** scegliere il tipo di valutazione del cliente (solo test / solo BIA / completo) con la spiegazione di ciascun profilo **per** configurare i moduli attivi fin dalla creazione.

`StepProfileType` usa `PROFILE_CATEGORIES` da `constants/bia.js` per descrivere ogni opzione: `tests_only` (solo test atletici, ha rank), `bia_only` (solo BIA, nessun rank), `complete` (entrambi, rank da test). Il tipo di profilo è modificabile in seguito tramite upgrade. Questo step è presente solo nel wizard PT; in Soccer il profilo è sempre `tests_only`.

```
components/modals/new-client-wizard/steps/StepProfileType.jsx
```

US-095 — Step BIA iniziale (StepBia)

Come trainer **voglio** inserire i valori della prima misurazione BIA durante il wizard **per** avere i dati di composizione corporea disponibili subito dopo la creazione del cliente.

`StepBia` appare nel wizard solo per profili `bia_only`. Presenta tutti i parametri BIA con il BMI pre-calcolato automaticamente da peso e altezza inseriti nel primo step. I campi non obbligatori possono essere saltati. Alla conferma del wizard, i valori vengono salvati direttamente in `biaHistory` e `lastBia` del documento cliente.

```
components/modals/new-client-wizard/steps/StepBia.jsx
```

Piani SaaS e Limiti

US-079 — Piano attivo e limiti dell'organizzazione

Come `org_admin` **voglio** vedere il piano attivo e i limiti di trainer e clienti **per** capire i vincoli dell'organizzazione.

`OrgSettingsPage` mostra il piano corrente (free / pro / enterprise) con una select per cambiarlo e una descrizione dinamica dei limiti: free (1 trainer, 10 clienti), pro (5 trainer, 100 clienti), enterprise (illimitati). La modifica del piano aggiorna il documento org e i limiti vengono applicati immediatamente sia dalla UI che dalle Firestore rules.

```
features/org/org-pages/OrgSettingsPage.jsx
```

US-080 — Banner di blocco al raggiungimento del limite piano

Come `org_admin` **voglio** ricevere un banner di blocco quando raggiungo il limite del piano **per** essere informato prima di tentare di aggiungere oltre il consentito.

`NewClientView` mostra una schermata di blocco invece del wizard quando `clientCount >= getPlanLimits(plan).clients`. `MembersPage` mostra un banner giallo e disabilita il bottone AGGIUNGI quando `memberCount >= getPlanLimits(plan).trainers`. I limiti sono applicati in modo ridondante anche via Firestore rules (solo su operazioni `create`).

```
features/trainer/NewClientView.jsx , features/org/org-pages/MembersPage.jsx
```

Multi-Modulo

US-081 — UI adattata al modulo dell'organizzazione (PT o Soccer)

Come trainer **voglio** vedere terminologia, test e comportamento UI adattati al modulo della mia organizzazione **per** lavorare con un'interfaccia coerente con il dominio di riferimento.

`TrainerContext` espone `moduleType` e `terminology`. `getModule(moduleType).isSoccer` è la fonte di verità per tutte le ramificazioni condizionali nel codice. In Soccer Academy la terminologia sostituisce automaticamente "Cliente" con "Allievo", "Gruppo" con "Squadra", "Sessione" con "Allenamento" in tutta l'interfaccia. Il `moduleType` è immutabile per organizzazione dopo la creazione.

`config/modules.config.js`, `context/TrainerContext.jsx`

Documento generato dal codice sorgente — maggio 2026 · aggiornato con gap analysis e descrizioni estese